

EduVision

Student Performance Prediction System

Submitted by

Barkha Pathak
Nandini Pandey
Tushti Savarn

For the award of the degree of

MCA

Under the supervision of

Supervisor

Dr Manoj Sain, Assistant Professor, Banasthali Vidyapith



Department of Mathematics and Computing

Banasthali Vidyapith

Banasthali - 304022

Session: 2025-2026

Certificate

Certified that **Barka Pathak, Nandini Pandey, Tushti Savarn** has carried out the project work titled **EduVision** from **Dec 2025** to **April 2026** for the award of the **MCA** from **Banasthali Vidhyapith** under my supervision. The thesis embodies result of original work and studies carried out by Student herself and the contents of the thesis do not form the basis for the award of any other degree to the candidate or to anybody else.

Dr Manoj Sain

Designation: Assistant Professor

Place:

Date:

Abstract

EduVision is a web-based application designed to analyze and predict student academic performance using historical data such as marks, attendance, and assignment records. The system aims to help teachers identify students who may need additional support and improve overall academic outcomes. The application allows teachers to upload and manage student data, while the system processes this information to generate performance predictions.

Students can also view their academic details in a structured and transparent manner. By using data analysis techniques, the system provides insights that assist in early intervention and better decision-making.

The project is developed using Django for backend processing and HTML/CSS for frontend design, with a centralized database for efficient data management. The system reduces manual effort, improves accuracy, and enhances the ability to monitor student progress. Overall, the Student Performance Prediction System serves as a useful tool for educational institutions to predict performance trends, support students effectively, and improve academic efficiency.

Acknowledgement

The successful completion of this group project would not have been possible without the collective efforts and support of many individuals.

First and foremost, sincere gratitude is extended to our mentor, whose constant guidance, valuable suggestions, and encouragement helped us throughout the project. Their expertise and insights played a significant role in shaping our work and improving its quality.

We would also like to express our heartfelt thanks to our project guide for providing continuous support, constructive feedback, and direction at every stage of the project.

Special appreciation is extended to all the faculty members of the department for providing the necessary resources, knowledge, and a conducive learning environment to carry out this project successfully. We are grateful to our teammates for their cooperation, dedication, and teamwork. The successful completion of this project is a result of our combined efforts, coordination, and mutual support.

Table of Contents

1. Title Page
2. Submitted By
3. Supervisor Details
4. Certificate
5. Abstract
6. Acknowledgement
7. Objective
8. Requirement Analysis (SRS)
 - Requirement Specification
 - Functional Requirements
 - Non-Functional Requirements
 - Hardware and Software Requirements
 - Feasibility Study
 - Technical Feasibility
 - Operational Feasibility
 - Economic Feasibility
 - Legal and Ethical Feasibility
 - Product Functions
9. System Design Specification (SDS)
 - Flowchart Diagram
 - ER Diagram
 - Sequential Diagram
10. Code Snippets
 - Frontend
 - Backend
 - ML Model & Output
 - Database
11. User Interface
12. References

Objective

The main objective of the Student Performance Prediction System is to develop a web-based application that analyzes student academic data and predicts their performance to support better academic decision-making.

- To digitize and manage student academic records efficiently:
- Replace manual record-keeping with a centralized system for storing and managing student data.
- Evaluate student performance using data such as marks, attendance, and assignment records.
- To predict student performance:
- Use past data patterns to estimate future academic performance of students.
- To identify weak students at an early stage:
- Help teachers recognize students who may require additional support and attention.
- Provide useful insights that help teachers plan better teaching strategies and interventions.
- To increase accuracy and reliability:
- Minimize human errors in managing and analyzing academic data.
- To provide easy access to information:
- Allow students and teachers to access academic data anytime through a web-based platform.
- To enhance transparency

Requirement Analysis (SRS)

Requirement Specification

The Software Requirement Specification (SRS) defines the complete description of EduVision , including its features, functionalities, constraints, and system behavior. It provides a clear understanding of how the system should operate and serves as a guideline for development and evaluation

Functional Requirements

Functional requirements define what the system is expected to do and the core features it provides to users.

User Login/Logout:

The system provides secure login and logout functionality for teachers and students to ensure authorized access.

Student Record Management:

Teachers can add, update, delete, and manage student details such as personal and academic information.

Marks Management:

The system allows teachers to upload, update, and manage student marks for different subjects.

Attendance Management:

Teachers can record and maintain student attendance, and the system stores it for tracking and analysis.

Assignment Management:

Teachers can upload assignments, and students can view them through the system.

Performance Prediction:

The system analyzes past data such as marks and attendance to predict student performance.

View Academic Details:

Students can view their marks, attendance, assignments, and predicted performance in a structured format.

Data Storage and Retrieval:

All data is stored in a centralized database, allowing easy access and efficient management.

Non-Functional Requirements

Non-functional requirements define how the system performs and the quality attributes it must maintain.

Performance:

The system should provide fast response times and handle multiple users efficiently without delay.

Security:

The system must ensure secure login authentication and protect sensitive student data from unauthorized access.

Usability:

The interface should be simple, user-friendly, and easy to navigate for both teachers and students.

Reliability:

The system should provide accurate results and maintain data consistency without errors.

Scalability:

The system should be able to handle increasing data and users and allow future enhancements.

Availability:

The system should be accessible whenever required with minimal downtime.

Maintainability:

The system should be easy to update, modify, and fix if any issues arise.

Hardware and Software Requirements

Hardware Requirements

The system does not require any specialized hardware and can run on standard computing devices.

Processor: Intel i3 or higher

RAM: Minimum 4 GB

Storage: At least 20 GB free disk space

Input Devices: Keyboard and Mouse

Output Device: Monitor

Network: Internet connection required

Software Requirements

The system is developed using commonly available software technologies.

Operating System: Windows / Linux

Frontend: HTML, CSS

Backend: Python (Django Framework)

Database: SQLite

Web Browser: Google Chrome, Microsoft Edge

Development Tools: Visual Studio Code

Feasibility Study

1. Technical feasibility

EduVision is technically feasible because it is built using widely available, reliable, and well-supported technologies.

Use of Open-Source Technologies

The system is developed using Python (Django), HTML, CSS, and JavaScript, which are free and open-source. This reduces dependency on paid tools.

Simple and Manageable Architecture

The project follows a standard web-based architecture (frontend + backend + database), making it easy to develop, understand, and maintain.

Database Support

The system uses a relational database (like SQLite/MySQL), which efficiently handles structured data such as students, marks, attendance, and assignments.

Availability of Development Tools

Tools like VS Code, Django framework, and web browsers are easily available and require no special setup.

Hardware Requirements are Minimal

The system can run on basic computers/laptops with standard configuration, making it accessible for most institutions.

Scalability

The system can be upgraded in the future by integrating machine learning models for better prediction accuracy or deploying on cloud platforms.

Integration Capability

It can be integrated with other systems such as student portals or learning management systems if required.

Security Features

Django provides built-in security features like authentication, authorization, and protection against common attacks (CSRF, SQL Injection).

2.Operational Feasibility

Operational feasibility determines whether the proposed system will function effectively in a real-world environment and be accepted by its users.

User-Friendly Interface:

The system is designed with a simple and easy-to-understand interface, making it convenient for both teachers and students to use.

Ease of Use:

Users require only basic computer and internet knowledge to operate the system, so no specialized technical skills are needed.

Reduced Manual Work:

The system automates tasks such as managing student records, tracking attendance, and analyzing performance, reducing paperwork and manual effort.

Time Efficiency:

Academic data can be accessed and updated quickly, saving time for both teachers and students.

Improved Accuracy:

Automation reduces human errors in calculations, data entry, and record maintenance.

Better Decision Making:

The system helps teachers analyze student performance and identify weak students, enabling timely support and improvement.

Minimal Training Required:

Since the system is simple, users can easily adapt to it without extensive training.

Accessibility:

Being a web-based system, it can be accessed anytime and from anywhere with an internet connection.

User Acceptance:

The system aligns with existing academic processes, making it easy for users to accept and adopt it.

3.Economic Feasibility

Economic feasibility evaluates whether the proposed system is cost-effective and financially viable.

Low Development Cost:

The system is developed using open-source technologies such as Python (Django), HTML, and CSS, which eliminates licensing costs.

No Expensive Hardware Required:

The system can run on basic computers or laptops, so there is no need for high-end hardware investment.

Minimal Software Cost:

Tools used for development (like code editors and browsers) are freely available, reducing overall expenses.

Low Maintenance Cost:

The system is easy to maintain and update, requiring minimal financial resources over time.

Reduced Manual Work:

Automation of tasks like managing student data, attendance, and performance analysis reduces the need for manual labor, saving operational costs.

Time Saving & Cost Saving:

Faster data processing and easy access to information reduce time consumption, indirectly lowering costs.

Improved Accuracy:

Reduction in human errors minimizes the cost associated with correcting mistakes.

4. Legal and Ethical Feasibility

Legal and ethical feasibility ensures that the system complies with laws, regulations, and ethical standards, especially in handling user data.

Data Privacy:

The system ensures that student information such as marks, attendance, and personal details are kept secure and confidential.

User Authorization:

Only authorized users (teachers and students) can access the system through secure login credentials, preventing unauthorized access.

Compliance with Rules:

The system follows general data protection guidelines and institutional policies for handling academic records.

No Use of Pirated Software:

The project is developed using open-source technologies, ensuring legal compliance and avoiding copyright issues.

Data Security:

Proper measures such as authentication and controlled access are used to protect sensitive data from misuse.

Transparency:

Students can view their own data, ensuring transparency in marks and performance evaluation.

Fairness in Prediction:

The system provides predictions based on available data without bias, supporting fair academic decisions.

Product Function

The Student Performance Prediction System is a feasible product as it directly addresses the problem of manual student performance analysis and lack of early prediction in educational institutions.

Solves a Real Problem

The system helps in predicting student performance, which is usually not done in traditional systems. It allows early identification of weak students.

Useful for Multiple Users

The product is beneficial for teachers (analysis & monitoring), students (self-evaluation), and institutions (better academic planning).

Improves Decision Making

Teachers can take timely actions such as extra classes or guidance for students who are at risk.

Reduces Manual Work

Automates tasks like marks management, attendance tracking, and performance analysis.

User-Friendly Design

The interface is simple, so users with basic technical knowledge can easily use the system.

Accurate and Reliable Output

Predictions are based on stored academic data, which improves accuracy compared to manual estimation.

Supports Academic Growth

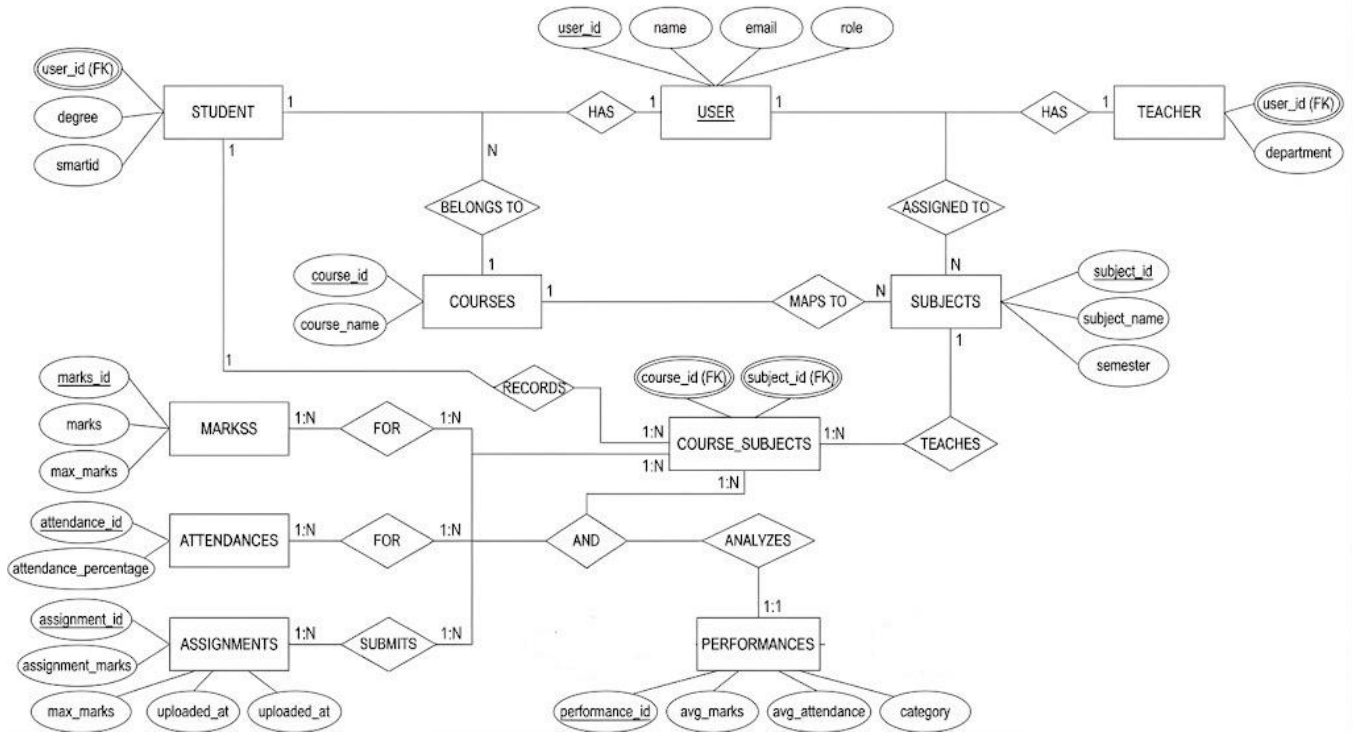
Helps in improving overall student performance and institutional results.

Expandable Product

Future enhancements like AI/ML models, dashboards, and analytics can make the product even more powerful.

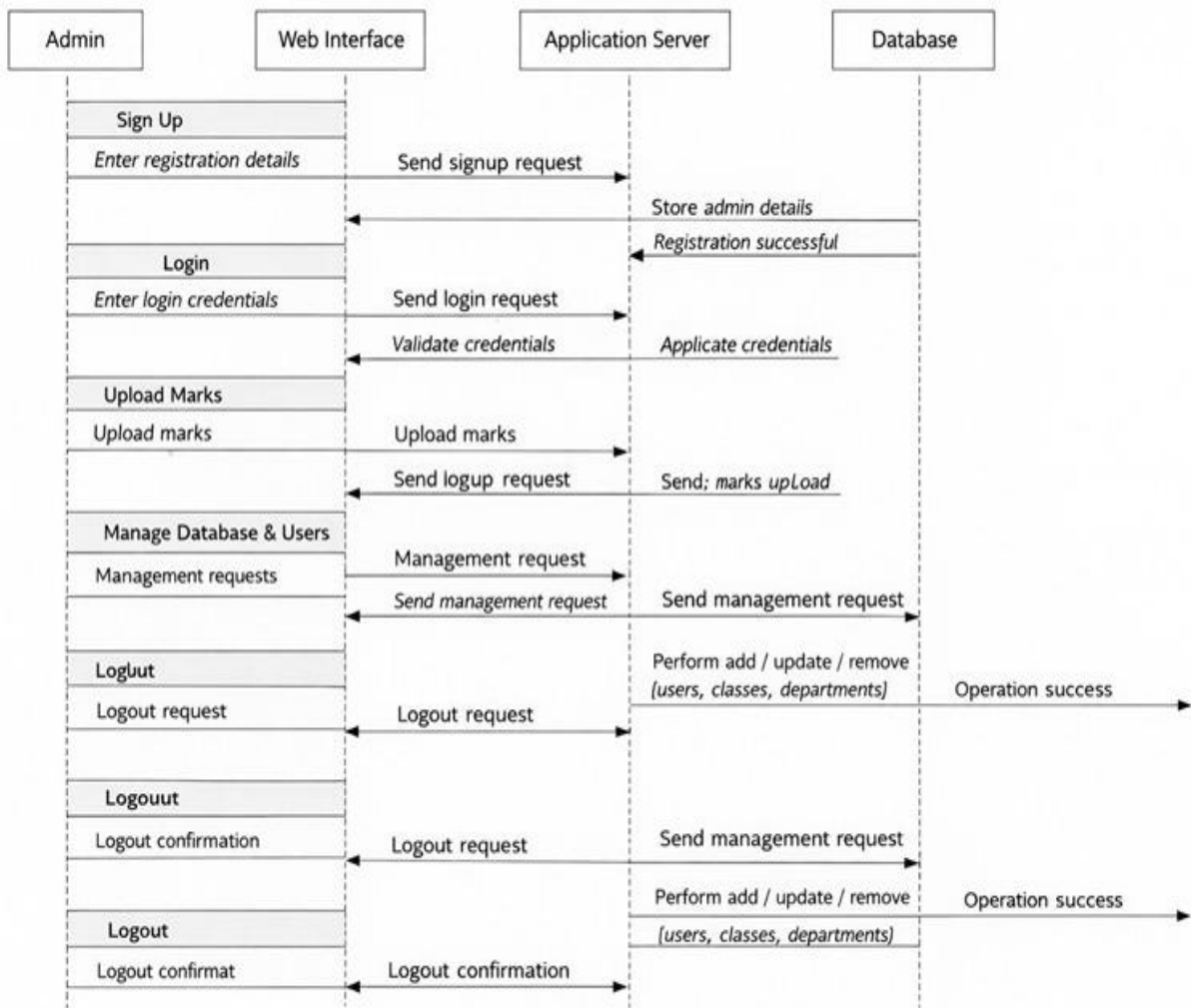
System Design Specification (SDS)

ER DIAGRAM

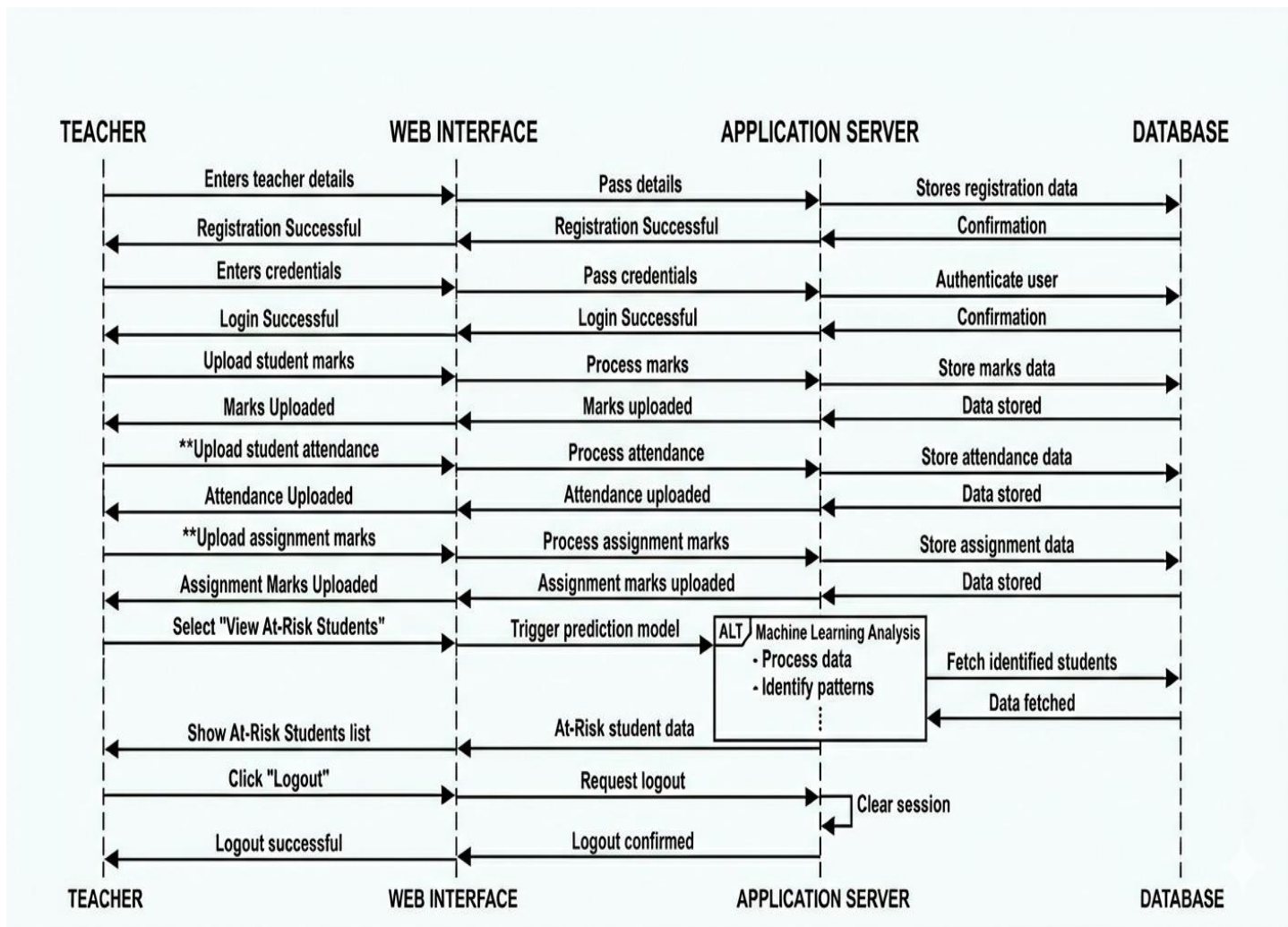


SEQUENTIAL DIAGRAM

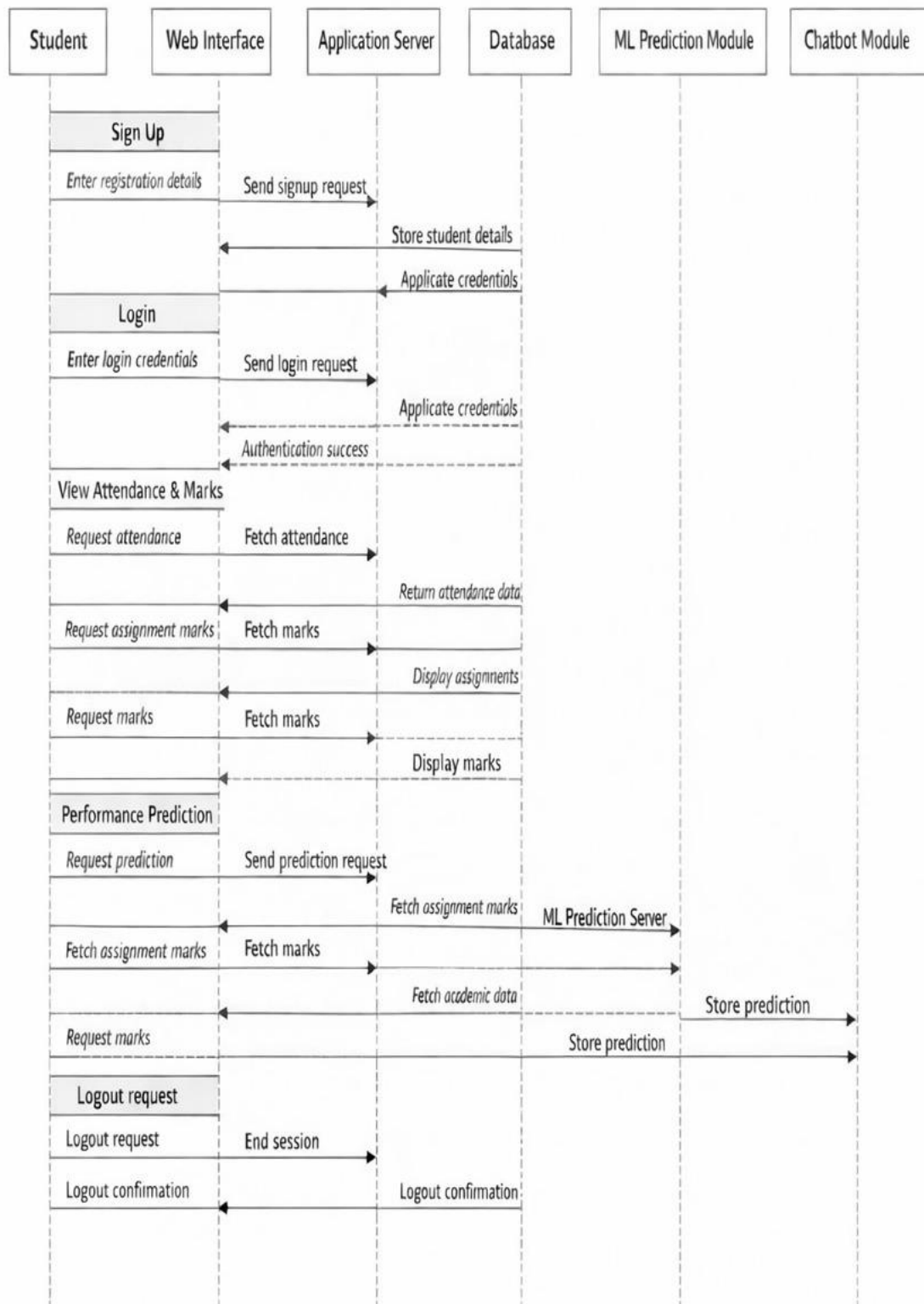
Admin sequence diagram



Teacher sequence diagram



Student sequence diagram



Code Snippets:

Frontend:

```

head>
  <style>
  </style>
/head>
body>
  <div class="main-wrapper">
    <header class="navbar">
      <div class="logo">
        
        <span>EduVision</span>
      </div>
      <nav>
        <div class="dropdown">
          <span class="nav-btn">Signup ▼</span>
          <div class="dropdown-content">
            <a href="{% url 'stu_signup' %}">Student</a>
            <a href="{% url 'teacher_signup' %}">Teacher</a>
          </div>
        </div>
      </nav>
    </header>

    <main class="content-area">
      <section class="hero-split">
        <div class="hero-text">
          <h1>Unlock Your <span class="highlight">Academic Potential</span></h1>
          <p>Predict your performance, identify gaps, and excel before you even step into the exam hall.</p>
          <button class="predict-btn-lg">Get Started</button>
        </div>
      </section>
    </main>
  </div>

```

```

<body>
<div class="container">
  <div class="card">
    <h3>📊 Comparative Analytics</h3>
    <canvas id="studentChart" height="100"></canvas>
  </div>

  <div class="card table-wrapper">
    <h3 style="padding: 20px 20px 0;">Subject Breakdown</h3>
    <table>
      <thead>
        <tr>
          <th onclick="sortTable(0)">Subject ⬆</th>
          <th onclick="sortTable(1)">Marks ⬆</th>
          <th onclick="sortTable(2)">Attendance ⬆</th>
          <th onclick="sortTable(3)">Assignment ⬆</th>
        </tr>
      </thead>
      <tbody>
        {% for d in data %}
        <tr class="{% if d.marks < 40 %}weak{% elif d.marks > 75 %}strong{% endif %}">
          <td>{{ d.subject }}</td>
          <td>{{ d.marks }}%</td>
          <td>{{ d.attendance }}%</td>
          <td>{{ d.assignment }}%</td>
        </tr>
        {% endfor %}
      </tbody>
    </table>
  </div>
</div>

```

```

<html lang="en">
<body>
<div class="container">

  <div class="predict-container">
    <form method="POST">
      {% csrf_token %}
      <select name="course_subject_id" required>
        <option value="">-- Select Subject to Analyze --</option>
        {% for cs in course_subjects %}
          <option value="{{ cs.id }}">
            {{ cs.course.course_name }} - {{ cs.subject.subject_name }}
          </option>
        {% endfor %}
      </select>
      <button type="submit" class="predict-btn">🚀 Analyze Performance</button>
    </form>
  </div>

  {% if show_result %}
  <div id="resultSection" style="margin-top: 50px; animation: fadeUp 0.8s ease;">
    <h2 style="text-align: center; margin-bottom: 10px;">📊 Analysis Report</h2>
    <p style="text-align: center; color: #9fb3d9; margin-bottom: 30px;">
      Real-time classification based on latest academic data.
    </p>

    <canvas id="performanceChart"></canvas>

    <div class="result-section">
      <div class="result-box excellent-box">

```


Backend:

```
# =====
# 🔒 AUTH DECORATORS
# =====

def teacher_required(view_func):
    @wraps(view_func)
    def _wrapped(request, *args, **kwargs):
        if not request.user.is_authenticated:
            return redirect('login')
        if not Teacher.objects.filter(user=request.user).exists():
            return HttpResponseForbidden("Teachers only")
        return view_func(request, *args, **kwargs)
    return _wrapped

def student_required(view_func):
    @wraps(view_func)
    def _wrapped(request, *args, **kwargs):
        if not request.user.is_authenticated:
            return redirect('login')
        if not Student.objects.filter(user=request.user).exists():
            return HttpResponseForbidden("Students only")
        return view_func(request, *args, **kwargs)
    return _wrapped

# =====
# 🔒 LOGIN
# =====
```

```
from django.urls import path
from . import views
from django.contrib import admin

urlpatterns = [

    path('', views.home, name='home'),
    path('login/', views.login_view, name='login'),
    path('logout/', views.logout_view, name='logout'),

    path('student-signup/', views.student_signup, name='stu_signup'),
    path('teacher-signup/', views.teacher_signup, name='teacher_signup'),
    path('student-dashboard/', views.student_dashboard, name='student_dashboard'),
    path('teacher-dashboard/', views.teacher_dashboard, name='teacher_dashboard'),

    path('upload-marks/', views.upload_marks, name='upload_marks'),
    path('upload-attendance/', views.upload_attendance, name='upload_attendance'),
    path('upload-assignment/', views.upload_assignment, name='upload_assignment'),
    path('chatbot/', views.chatbot_response, name='chatbot'),
    path('predict/', views.teacher_dashboard, name='predict'),

    path('admin/', admin.site.urls),
```

Database:

```
from django.db import models
from django.contrib.auth.models import User

# STUDENT MODEL
class Student(models.Model):
    user = models.OneToOneField(User, on_delete=models.CASCADE)
    degree = models.CharField(max_length=100)
    smartid = models.CharField(max_length=50)

    def __str__(self):
        return self.user.username

# TEACHER MODEL
class Teacher(models.Model):
    user = models.OneToOneField(User, on_delete=models.CASCADE)
    department = models.CharField(max_length=100)

    def __str__(self):
        return self.user.username

# ● COURSE
class Course(models.Model):
    course_name = models.CharField(max_length=100)
```


● COURSE

```
class Course(models.Model):  
    course_name = models.CharField(max_length=100)  
  
    def __str__(self):  
        return self.course_name
```

● SUBJECT

```
class Subject(models.Model):  
    subject_name = models.CharField(max_length=100)  
  
    def __str__(self):  
        return self.subject_name
```

● COURSE-SUBJECT

```
class CourseSubject(models.Model):  
    course = models.ForeignKey(Course, on_delete=models.CASCADE)  
    subject = models.ForeignKey(Subject, on_delete=models.CASCADE)  
  
    def __str__(self):  
        return f"{self.course.course_name} - {self.subject.subject_name}"
```

● TEACHER ASSIGNMENT

```
class TeacherAssignment(models.Model):  
    teacher = models.ForeignKey(Teacher, on_delete=models.CASCADE)
```

ML Model:

```

# ⚠️ IMPORTANT: Ensure dataset.csv is in SAME folder
df = pd.read_csv("dataset.csv")

print("Dataset Loaded Successfully!\n")
print(df.head())

# =====
# 🧹 DATA CLEANING
# =====

required_columns = ["attendance", "periodical", "assignment", "label"]
df = df[required_columns]

df.dropna(inplace=True)

# Clip values
df["attendance"] = df["attendance"].clip(0, 100)
df["periodical"] = df["periodical"].clip(0, 100)
df["assignment"] = df["assignment"].clip(0, 100)

# =====
# 📊 LABEL DISTRIBUTION
# =====

plt.figure()
sns.countplot(x="label", data=df)
plt.title("Label Distribution")
plt.show()

# -----

```

```

X = df[["attendance", "periodical", "assignment"]]
y = df["label"]

# Encode labels
encoder = LabelEncoder()
y_encoded = encoder.fit_transform(y)

print("\nLabel Mapping:")
for i, label in enumerate(encoder.classes_):
    print(f"{label} -> {i}")

# =====
# 📉 CORRELATION
# =====

plt.figure()
sns.heatmap(X.corr(), annot=True)
plt.title("Feature Correlation")
plt.show()

# =====
# ✂️ TRAIN TEST SPLIT
# =====

X_train, X_test, y_train, y_test = train_test_split(
    X, y_encoded,
    test_size=0.2,
    random_state=42,

```

```

y_pred = rf_model.predict(X_test)

accuracy = accuracy_score(y_test, y_pred)

print("\nModel Accuracy:", round(accuracy * 100, 2), "%")

print("\nClassification Report:\n")
print(classification_report(y_test, y_pred, target_names=encoder.classes_))

# =====
# 📊 CONFUSION MATRIX
# =====

cm = confusion_matrix(y_test, y_pred)

plt.figure()
sns.heatmap(cm, annot=True, fmt="d",
            xticklabels=encoder.classes_,
            yticklabels=encoder.classes_)
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Confusion Matrix")
plt.show()

# =====
# 🔄 CROSS VALIDATION
# =====

```

```

▽ feature_df = pd.DataFrame({
    "Feature": X.columns,
    "Importance": rf_model.feature_importances_
}).sort_values(by="Importance", ascending=False)

print("\nFeature Importance:\n")
print(feature_df)

plt.figure()
sns.barplot(x="Importance", y="Feature", data=feature_df)
plt.title("Feature Importance")
plt.show()

▽ models = {
    "Logistic Regression": LogisticRegression(max_iter=1000),
    "Decision Tree": DecisionTreeClassifier(),
    "SVM": SVC(),
    "Random Forest": rf_model
}

print("\nModel Comparison:\n")

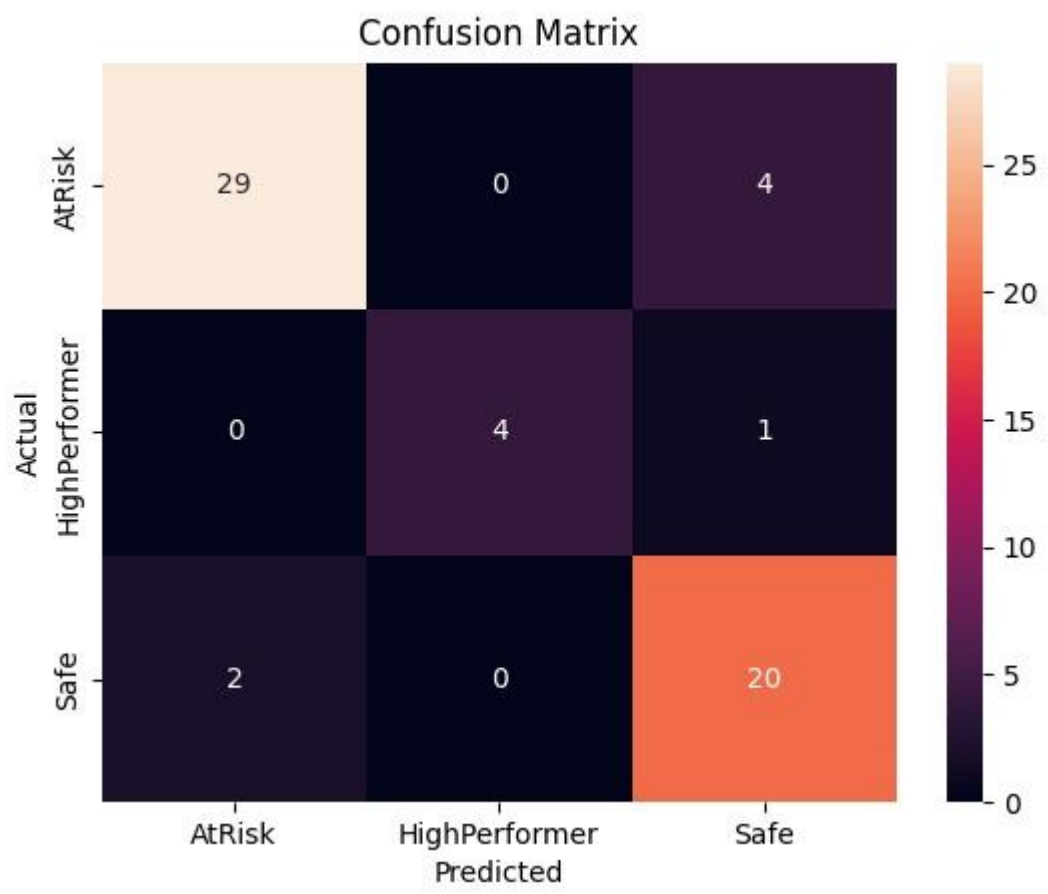
▽ for name, model in models.items():
    model.fit(X_train, y_train)
    preds = model.predict(X_test)
    acc = accuracy_score(y_test, preds)

    print(f"{name}: {round(acc * 100, 2)} %")

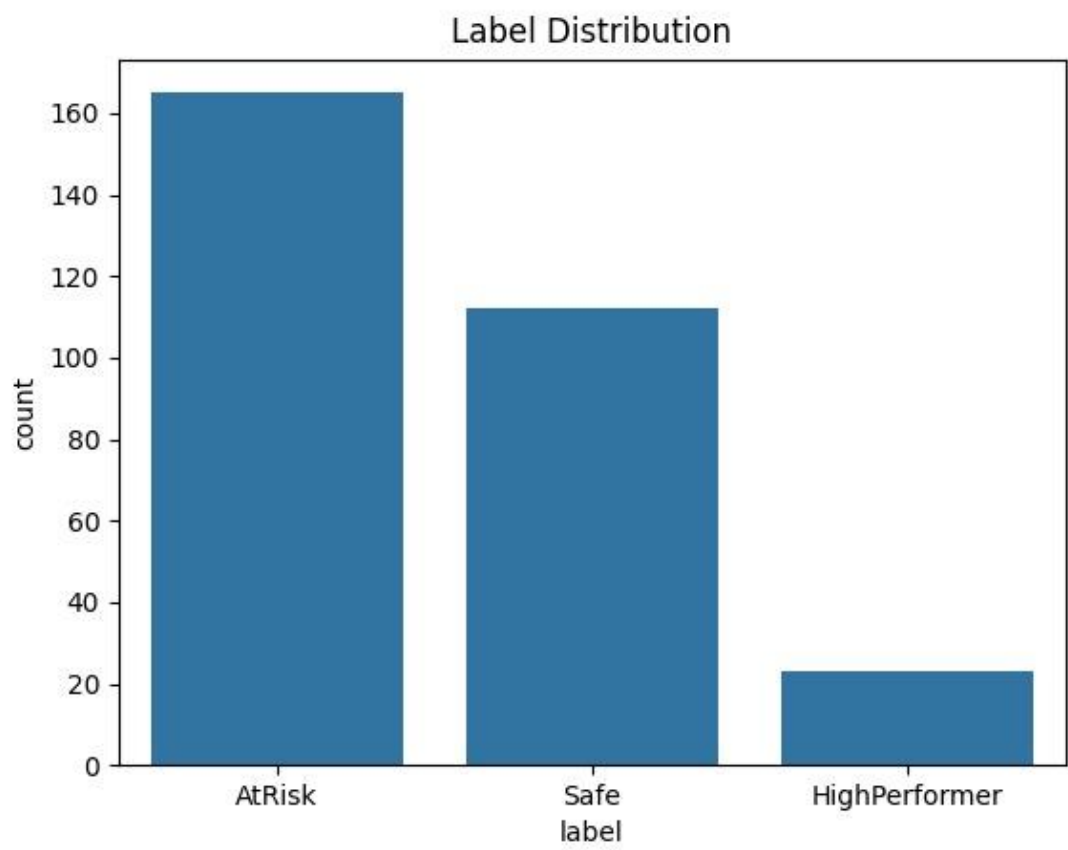
```

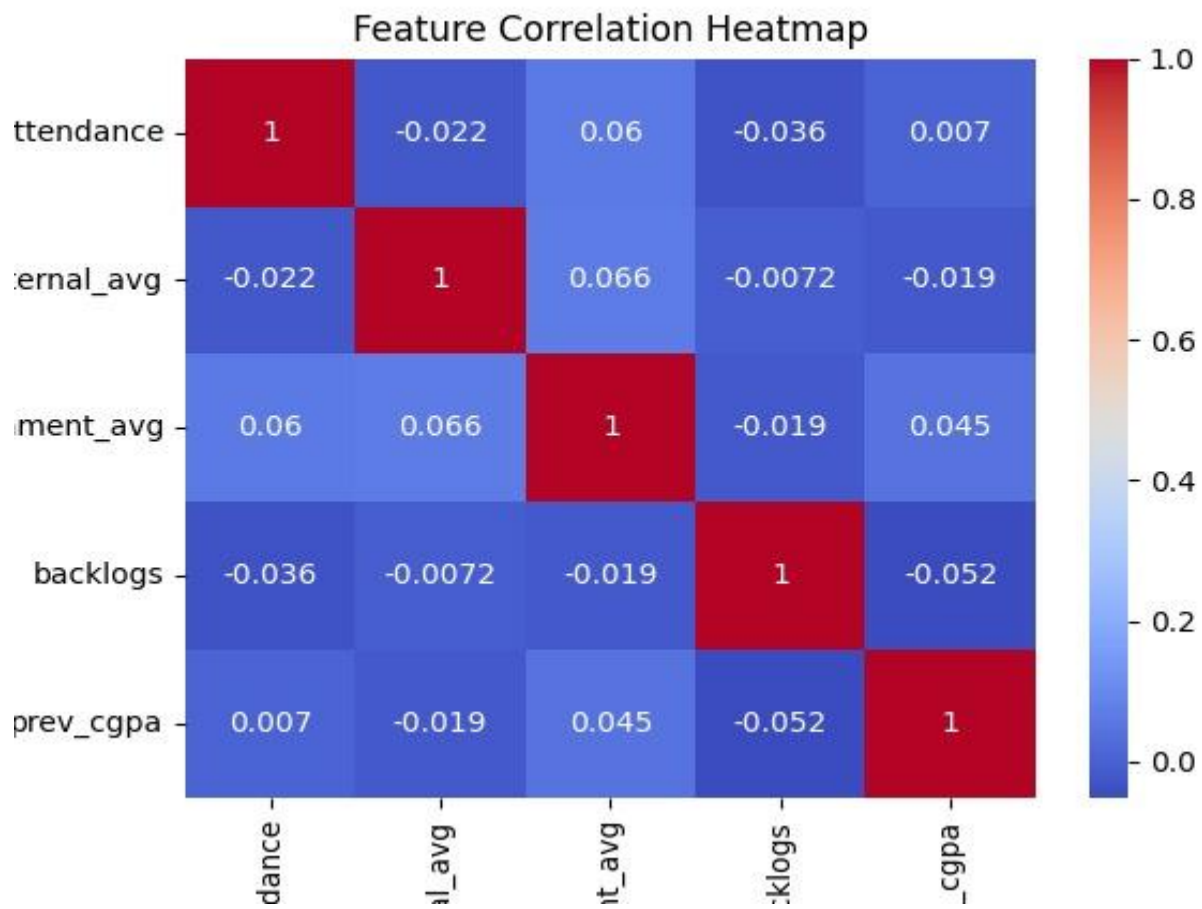

ML Outputs

Confusion Matrix

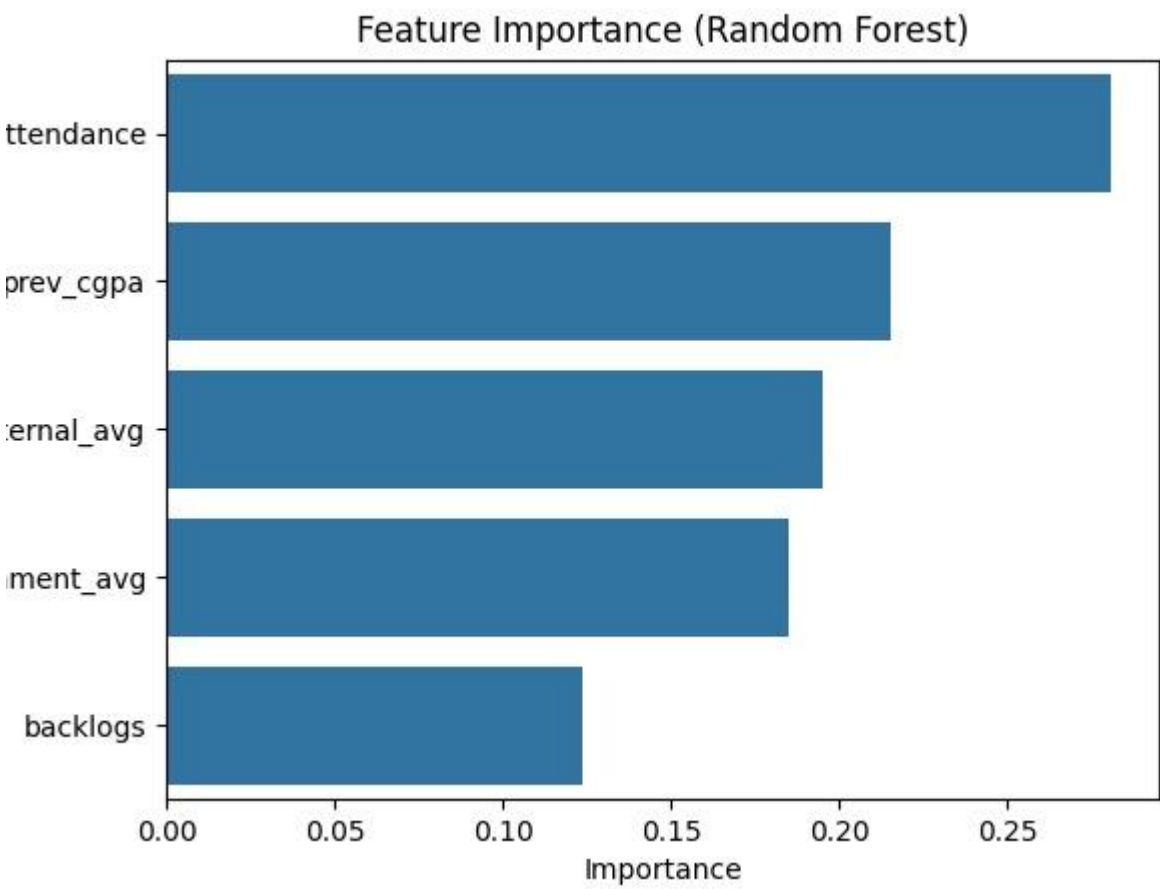


Label Distribution

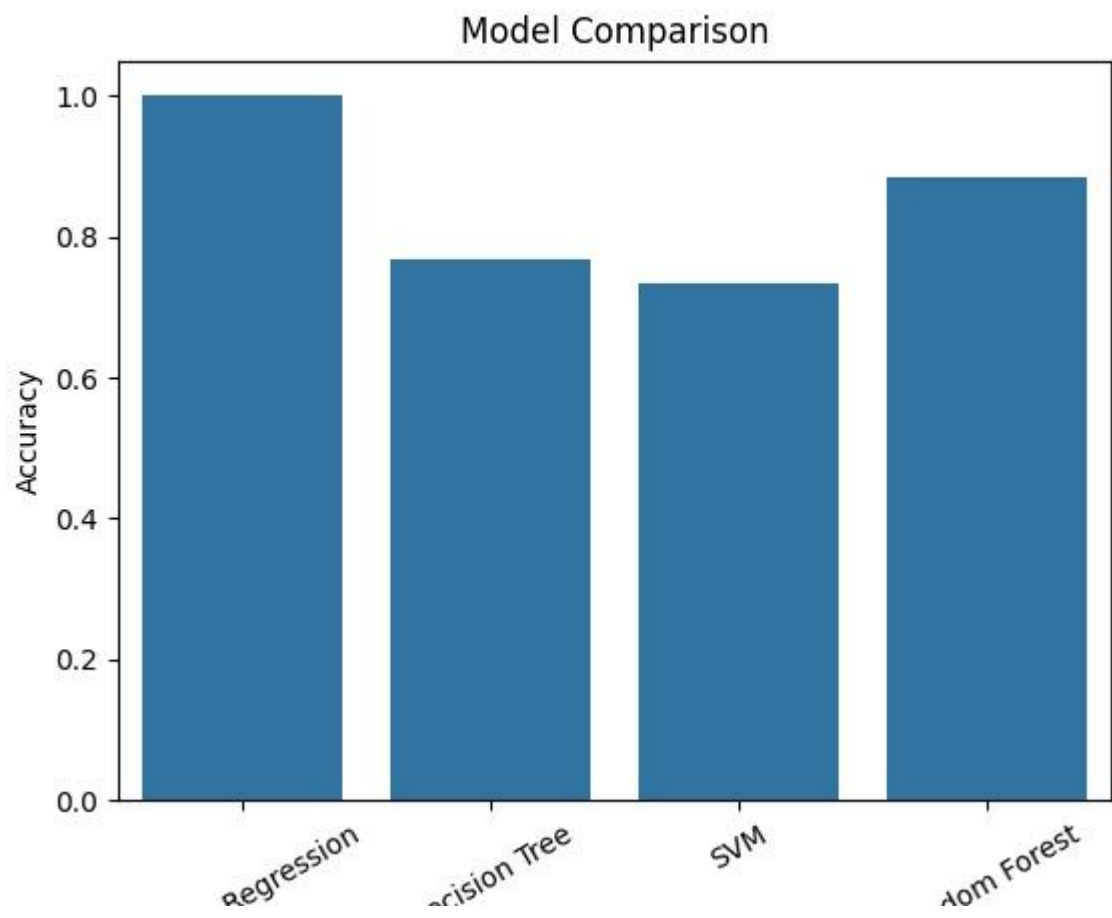


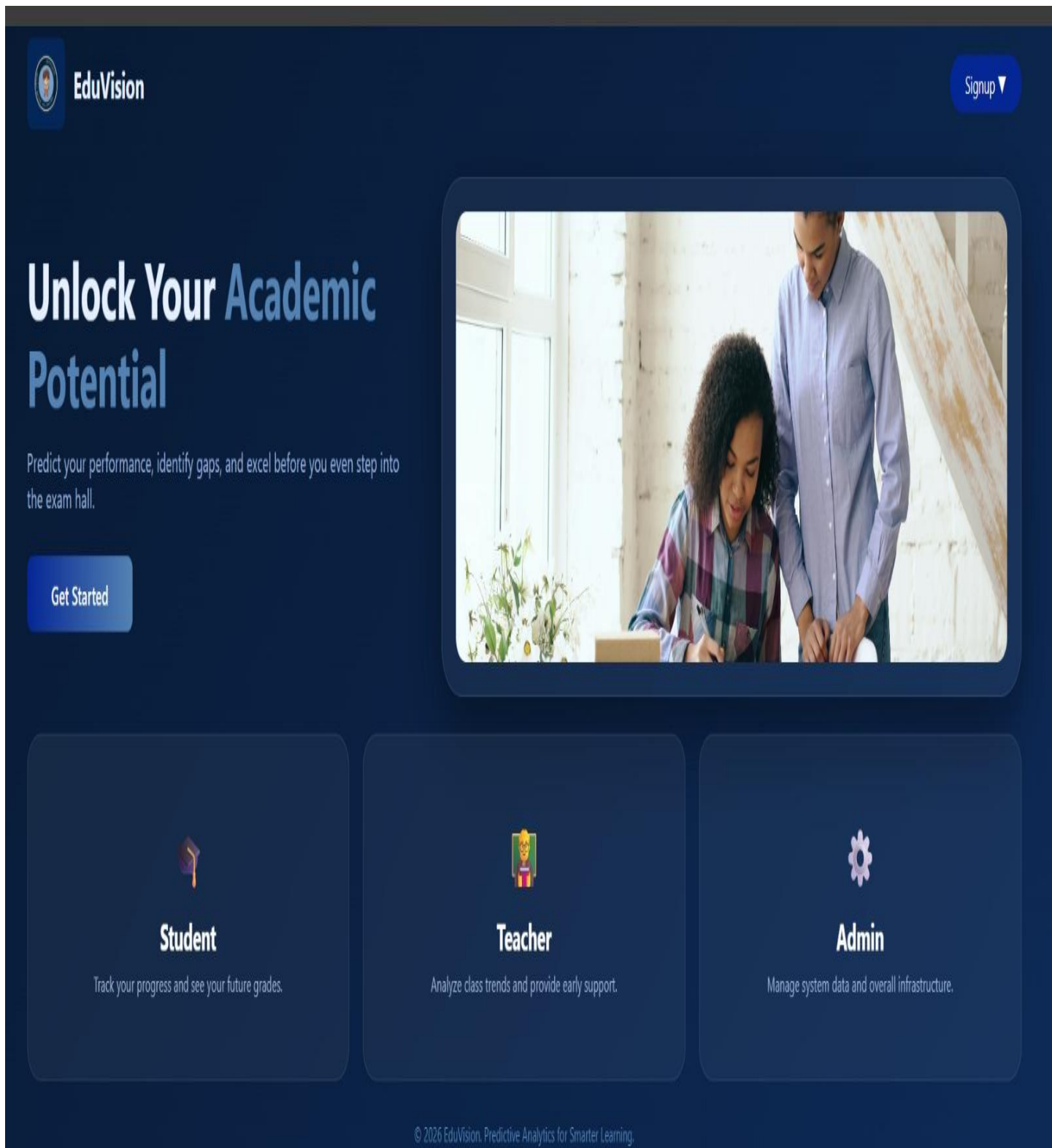
Feature Correlation heatmap

Feature Importance



Model Comparision



User Interface:

Hi, Sudha 🖐️

Upload student records and generate predictive analytics.

🌄 Good Morning!



📁 Upload Marks

Select Course ▼

Select Subject ▼

📅 Upload Attendance

Select Course ▼

Select Subject ▼

📄 Upload Assignment

Select Course ▼

Select Subject ▼

-- Select Subject to Analyze -- ▼

🚀 Analyze Performance



Excellent

- Rakhi
- Sona
- Kajal

Average

- kirti
- Bhavya
- Khushi
- Sandhya
- Rashi

Needs Attention

Rekha: Low Exam Marks

Hi, Rakhi 🙌

Good Morning

Email Id: rakhi@gmail.com



- DSA
- DBMS

Academic Standing

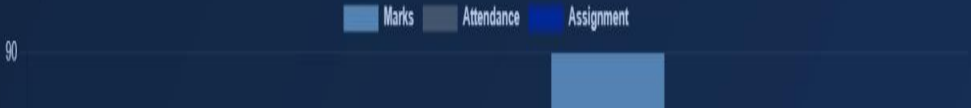
EXCELLENT

🏆 Rank: 8 | Avg: 70.2%

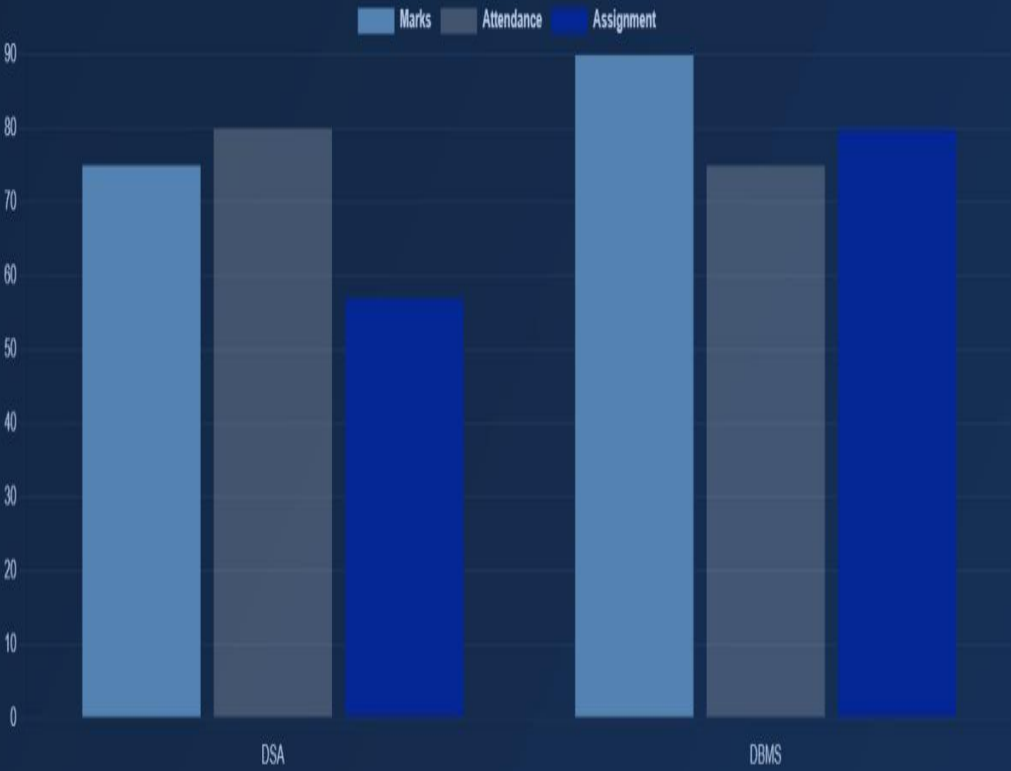
AI Insight

Good Performance

Comparative Analytics



Comparative Analytics



Subject Breakdown

Subject ↓	Marks ↓	Attendance ↓	Assignment ↓
DSA	75%	80.0%	57.0%
DBMS	90%	75.0%	80.0%



4 .References

Python Software Foundation. *Python Documentation*. Python Software Foundation, <https://www.python.org/doc/>.

Django Software Foundation. *Django Official Documentation*. Django Software Foundation, <https://docs.djangoproject.com/>.

World Wide Web Consortium. *HTML Specification*. W3C, <https://html.spec.whatwg.org/>.

World Wide Web Consortium. *CSS Specification*. W3C, <https://www.w3.org/Style/CSS/>.

Mozilla Developer Network. *JavaScript Documentation*. MDN Web Docs, <https://developer.mozilla.org/en-US/docs/Web/JavaScript>.

Mozilla Developer Network. *HTML & CSS Documentation*. MDN Web Docs, <https://developer.mozilla.org/>.